บทที่ 6: การแบ่งข้อมูลเป็นส่วน ๆ (Partitioning)

(สรุปเป็นภาษาไทยจากฉบับที่ 1 ใช้เวลาฟังประมาณ 15 นาที)

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ Designing Data-Intensive Applications บทที่หก ว่าด้วยการแบ่งข้อมูลเป็นส่วน ๆ หรือ Partitioning

ในบทที่แล้ว เราสมมติว่าข้อมูลทั้งหมดใส่ในเครื่องเดียวได้ แล้วทำสำเนาไปไว้หลายเครื่อง แต่ถ้าข้อมูลใหญ่มาก หรือมีคำขอเข้ามามากเกินกว่าที่เครื่องเดียวจะรับไหว ก็ต้องแบ่งข้อมูลออกเป็นส่วน ๆ แล้วกระจายไปไว้หลายเครื่อง

แต่ละส่วนเรียกว่า partition และแต่ละระบบก็ตั้งชื่อต่างกัน เช่น MongoDB และ Elasticsearch เรียกว่า shard, HBase เรียกว่า region, Bigtable เรียกว่า tablet ส่วน Cassandra เรียกว่า vnode แต่ทั้งหมดคือแนวคิดเดียวกัน

ข้อมูลแต่ละชิ้นอยู่ใน partition เดียวเท่านั้น และแต่ละ partition ก็เปรียบเหมือนฐานข้อมูลเล็ก ๆ ในตัวเอง เหตุผลหลักที่ต้องแบ่งคือการขยายระบบ เมื่อข้อมูลกระจายไปหลายเครื่อง ภาระงานก็กระจายตามไปด้วย

ในทางปฏิบัติ การแบ่งข้อมูลมักทำคู่กับการทำสำเนา หมายความว่าแต่ละ partition ก็มีสำเนาอยู่หลายเครื่อง และเครื่องหนึ่งเครื่องก็อาจเก็บหลาย partition เช่น เป็นผู้นำของ partition หนึ่ง และเป็นผู้ตามของอีก partition หนึ่ง

บทนี้จะพูดถึงสี่เรื่อง คือ วิธีแบ่งข้อมูล การจัดการดัชนีรอง การย้ายข้อมูลเมื่อเพิ่มหรือลดเครื่อง และวิธีส่งคำขอไปให้ถูกเครื่องครับ

ตอนที่ 2: เป้าหมายของการแบ่งข้อมูล

เป้าหมายคือกระจายข้อมูลและภาระงานให้เท่า ๆ กันทุกเครื่อง ถ้าทำได้ดี สิบเครื่องก็ควรรับงานได้ประมาณสิบเท่าของเครื่องเดียว

ถ้ากระจายไม่เท่ากัน บางเครื่องได้งานมากกว่าเครื่องอื่น เรียกว่า skew หรือความเบ้ ถ้าหนักจนงานเกือบทั้งหมดไปกองอยู่ที่ partition เดียว partition นั้นเรียกว่า hot spot หรือจุดร้อน เครื่องที่เหลือก็ว่างเปล่าโดยไม่มีประโยชน์

วิธีที่ง่ายที่สุดคือสุ่มส่งข้อมูลไปเครื่องไหนก็ได้ ข้อมูลจะกระจายเท่ากันดี แต่มีปัญหาใหญ่ คือเวลาจะอ่าน เราไม่รู้ว่าข้อมูลอยู่เครื่องไหน ต้องถามทุกเครื่อง จึงต้องหาวิธีที่ดีกว่านี้

ตอนที่ 3: แบ่งตามช่วงของคีย์

วิธีแรกคือแบ่งตามช่วงของคีย์ หรือ key range เหมือนสารานุกรมที่แยกเป็นหลายเล่ม เล่มแรกเก็บคำที่ขึ้นต้นด้วย A ถึง B เล่มที่สองเก็บ C ถึง D ไปเรื่อย ๆ ถ้ารู้ว่าแต่ละเล่มครอบคลุมช่วงไหน ก็หยิบเล่มที่ถูกต้องได้ทันที

ช่วงของแต่ละ partition ไม่จำเป็นต้องกว้างเท่ากัน เพราะข้อมูลไม่ได้กระจายเท่ากัน เช่น คำที่ขึ้นต้นด้วยตัว S อาจมีมากกว่าตัว X หลายเท่า ช่วงจึงต้องปรับให้ปริมาณข้อมูลใกล้เคียงกัน ระบบที่ใช้วิธีนี้ เช่น Bigtable, HBase และ RethinkDB

ข้อดีคือ ใน partition เดียวกัน คีย์ถูกเรียงไว้แล้ว การค้นหาเป็นช่วงจึงทำได้ง่ายมาก

แต่มีกับดักเรื่องจุดร้อน

สมมติเรามีเซนเซอร์หลายตัวส่งข้อมูลเข้ามาตลอดเวลา และใช้เวลาเป็นคีย์ ถ้าแบ่งตามวัน ข้อมูลของวันนี้ทั้งหมดก็จะไปอยู่ partition เดียว partition นั้นทำงานหนักมาก ส่วน partition ของวันก่อน ๆ แทบไม่มีอะไรทำ

วิธีแก้คือเอาชื่อเซนเซอร์ไปไว้หน้าเวลาในคีย์ ข้อมูลจะกระจายตามเซนเซอร์ก่อน แล้วค่อยเรียงตามเวลา ข้อเสียคือ ถ้าอยากได้ข้อมูลทุกเซนเซอร์ในช่วงเวลาหนึ่ง ต้องค้นแยกทีละเซนเซอร์

ตอนที่ 4: แบ่งตามค่าแฮชของคีย์

วิธีที่สองคือนำคีย์ไปผ่านฟังก์ชันแฮชก่อน ฟังก์ชันแฮชจะเปลี่ยนคีย์เป็นตัวเลขที่ดูเหมือนสุ่ม แม้คีย์สองตัวจะต่างกันแค่นิดเดียว ตัวเลขที่ได้ก็ต่างกันมาก จากนั้นแบ่ง partition ตามช่วงของตัวเลขนี้แทน ข้อมูลจึงกระจายได้เท่ากันดี

ฟังก์ชันแฮชที่ใช้ไม่จำเป็นต้องแข็งแรงด้านความปลอดภัย เช่น Cassandra และ MongoDB ใช้ MD5 แต่มีข้อควรระวังคือฟังก์ชันแฮชที่ติดมากับบางภาษา เช่น hashCode ของ Java ไม่เหมาะ เพราะคีย์เดียวกันอาจได้ค่าต่างกันเมื่ออยู่คนละโปรแกรม

ระหว่างนี้ ผู้เขียนพูดถึงคำว่า consistent hashing ด้วย คำนี้หลายคนใช้ไม่ตรงความหมายเดิม จึงสร้างความสับสนได้ง่าย ผู้เขียนแนะนำให้เรียกว่า hash partitioning ดีกว่า

ราคาที่ต้องจ่าย

ราคาของวิธีนี้คือ การค้นหาเป็นช่วงทำได้ไม่ดี เพราะคีย์ที่เคยอยู่ติดกัน ถูกกระจายไปคนละ partition เช่น MongoDB ถ้าใช้การแบ่งแบบแฮช การค้นหาเป็นช่วงต้องส่งไปถามทุก partition ส่วนบางระบบ ก็ไม่รองรับการค้นหาเป็นช่วงด้วยคีย์หลักเลย

ทางสายกลางของ Cassandra

Cassandra ใช้คีย์ที่มีหลายส่วน ส่วนแรกใช้แฮชเพื่อเลือก partition ส่วนที่เหลือใช้เรียงข้อมูลภายใน partition นั้น

ตัวอย่างเช่น ในโซเชียลมีเดีย ถ้าใช้คีย์เป็นรหัสผู้ใช้ตามด้วยเวลาที่โพสต์ โพสต์ทั้งหมดของผู้ใช้หนึ่งคนจะอยู่ partition เดียวกัน และเรียงตามเวลาไว้แล้ว อยากดูโพสต์ของผู้ใช้คนหนึ่งในช่วงเวลาหนึ่ง ก็ค้นได้รวดเร็ว ขณะที่ผู้ใช้แต่ละคนก็ยังกระจายไปหลายเครื่อง

ตอนที่ 5: เมื่อมีคีย์ที่ร้อนจัด

แม้จะใช้แฮช ก็ยังหนีจุดร้อนไม่ได้ทั้งหมด ถ้าคำขอทั้งหมดมุ่งไปที่คีย์เดียวกัน ก็ต้องไปที่ partition เดียวกันอยู่ดี

ตัวอย่างคือคนดังในโซเชียลมีเดียที่มีผู้ติดตามหลายล้านคน พอเขาโพสต์อะไร ก็มีคนแห่กันมาแสดงความคิดเห็นที่โพสต์เดียวกัน ซึ่งใช้คีย์เดียวกัน

ระบบส่วนใหญ่แก้ปัญหานี้ให้อัตโนมัติไม่ได้ ต้องให้โค้ดของแอปจัดการเอง วิธีง่าย ๆ คือเติมเลขสุ่มสองหลักต่อท้ายคีย์ คีย์เดียวจึงกลายเป็นหนึ่งร้อยคีย์ ที่กระจายไปหลาย partition

ข้อเสียคือ เวลาอ่าน ต้องดึงจากทั้งหนึ่งร้อยคีย์มารวมกัน และต้องจดไว้ว่าคีย์ไหนถูกแยกแบบนี้ เพราะถ้าทำกับทุกคีย์ ซึ่งส่วนใหญ่ไม่ได้ร้อน ก็จะเปลืองโดยใช่เหตุ

ตอนที่ 6: การแบ่งดัชนีรอง

ที่ผ่านมาเราพูดถึงการค้นหาด้วยคีย์หลัก แต่ในบทที่สามเราเจอดัชนีรอง ซึ่งใช้ค้นจากข้อมูลอื่น เช่น หารถทุกคันที่เป็นสีแดง ดัชนีรองไม่ได้ผูกกับ partition ใดอย่างลงตัว จึงต้องเลือกวิธีแบ่ง ซึ่งมีสองแบบ

แบบแรก ดัชนีประจำแต่ละ partition

หนังสือยกตัวอย่างเว็บขายรถมือสอง แต่ละคันมีรหัส และข้อมูลถูกแบ่งตามรหัสรถ

ในแบบนี้ แต่ละ partition มีดัชนีรองของตัวเอง ซึ่งดูแลเฉพาะรถที่อยู่ใน partition นั้น เช่น partition แรกรู้ว่ารถสีแดงของตัวเองมีคันไหนบ้าง แต่ไม่รู้เลยว่า partition อื่นมีรถสีแดงหรือไม่ ดัชนีแบบนี้เรียกว่า local index

ข้อดีคือการเขียนง่าย เพิ่มรถหนึ่งคัน ก็แก้แค่ partition เดียว

ข้อเสียคือการอ่าน ถ้าอยากหารถสีแดงทั้งหมด ต้องส่งคำถามไปทุก partition แล้วรวมคำตอบ วิธีนี้เรียกว่า scatter/gather หรือกระจายแล้วรวบรวม ซึ่งทำให้เกิดปัญหาความหน่วงช่วงหางที่เราพูดถึงในบทที่หนึ่ง คือขอแค่ partition เดียวตอบช้า คำตอบทั้งหมดก็ช้าตาม

ถึงอย่างนั้น ระบบจำนวนมากก็ใช้วิธีนี้ เช่น MongoDB, Cassandra, Elasticsearch และ SolrCloud

แบบที่สอง ดัชนีรวมที่แบ่งตามคำค้น

อีกทางคือสร้างดัชนีรวมของทั้งระบบ ซึ่งครอบคลุมรถทุกคัน แต่ดัชนีนี้ใหญ่เกินจะเก็บไว้เครื่องเดียว จึงต้องแบ่งเหมือนกัน โดยแบ่งตามคำค้น เช่น สีที่ขึ้นต้นด้วยตัว A ถึง R อยู่ partition แรก ตัว S ถึง Z อยู่ partition ที่สอง ดัชนีแบบนี้เรียกว่า global index

ข้อดีคือการอ่านเร็ว อยากหารถสีแดง ก็ไปถาม partition เดียวที่ดูแลคำว่าสีแดง ไม่ต้องถามทุกเครื่อง

ข้อเสียคือการเขียนช้าและซับซ้อนขึ้น เพราะรถหนึ่งคันมีหลายคุณสมบัติ ทั้งสี ยี่ห้อ และรุ่น ซึ่งดัชนีของแต่ละคุณสมบัติอาจอยู่คนละ partition การเพิ่มรถหนึ่งคันจึงต้องแก้หลายที่

ในทางปฏิบัติ การอัปเดตดัชนีแบบนี้จึงมักเป็นแบบไม่รอ เช่น ใน DynamoDB ดัชนีรวมมักอัปเดตภายในเสี้ยววินาที แต่ถ้าระบบมีปัญหา ก็อาจช้ากว่านั้น ช่วงนั้นผู้ใช้จะค้นไม่เจอข้อมูลที่เพิ่งเขียน

ตอนที่ 7: การย้ายข้อมูลเมื่อระบบเปลี่ยน

ระบบไม่ได้อยู่นิ่ง ภาระงานเพิ่มก็ต้องเพิ่มเครื่อง ข้อมูลโตก็ต้องเพิ่มดิสก์ และบางเครื่องก็เสียจนต้องให้เครื่องอื่นรับงานแทน การย้ายข้อมูลและภาระงานจากเครื่องหนึ่งไปอีกเครื่อง เรียกว่า rebalancing

การย้ายที่ดีต้องมีสามข้อ ข้อแรก หลังย้ายแล้วภาระงานต้องกระจายเท่า ๆ กัน ข้อสอง ระหว่างย้าย ระบบต้องยังรับการอ่านและเขียนได้ตามปกติ ข้อสาม ต้องย้ายข้อมูลให้น้อยที่สุด เพื่อไม่ให้เปลืองเครือข่ายและดิสก์

วิธีที่ไม่ควรใช้ คือ เอาค่าแฮชไปหารด้วยจำนวนเครื่อง

ฟังดูง่าย เช่น มีสิบเครื่อง ก็เอาค่าแฮชหารสิบ เศษเท่าไรก็ไปเครื่องนั้น

ปัญหาคือ พอเพิ่มเป็นสิบเอ็ดเครื่อง เศษของคีย์เกือบทุกตัวจะเปลี่ยน ข้อมูลเกือบทั้งหมดต้องย้ายเครื่อง ซึ่งเปลืองมาก

วิธีที่หนึ่ง กำหนดจำนวน partition ไว้คงที่

สร้าง partition ไว้เยอะกว่าจำนวนเครื่องมาก ๆ ตั้งแต่ต้น เช่น มีสิบเครื่อง แต่สร้างหนึ่งพัน partition แต่ละเครื่องดูแลประมาณร้อย partition

พอเพิ่มเครื่องใหม่ เครื่องใหม่ก็ขอ partition บางส่วนมาจากเครื่องเดิมแต่ละเครื่อง จนทุกเครื่องได้ส่วนแบ่งเท่า ๆ กันอีกครั้ง สิ่งที่ย้ายคือ partition ทั้งก้อน ส่วนการจับคู่ว่าคีย์ไหนอยู่ partition ไหนไม่ต้องเปลี่ยนเลย ระบบที่ใช้วิธีนี้ เช่น Riak, Elasticsearch และ Couchbase

ข้อยากคือต้องเลือกจำนวน partition ให้ดีตั้งแต่ต้น ถ้าน้อยไป ระบบโตแล้วแต่ละ partition จะใหญ่เกินไป ถ้ามากไป ก็เปลืองค่าจัดการ

วิธีที่สอง แบ่งแบบยืดหยุ่นตามขนาดข้อมูล

ถ้าแบ่งตามช่วงของคีย์ การกำหนดช่วงไว้ตายตัวไม่สะดวก เพราะถ้ากำหนดผิด ข้อมูลอาจไปกองอยู่ partition เดียว จึงมีอีกวิธีหนึ่ง คือ เมื่อ partition ใหญ่เกินขนาดที่กำหนด ก็แบ่งเป็นสองส่วน และเมื่อข้อมูลถูกลบจนเล็กลงมาก ก็รวมกับ partition ข้าง ๆ คล้ายการแบ่งหน้าใน B-Tree ที่เราเจอในบทที่สาม

ตัวอย่างเช่น HBase ค่าเริ่มต้นจะแบ่ง partition เมื่อมีขนาดถึง 10 กิโลไบต์ ขอแก้นะครับ 10 กิกะไบต์ ระบบอื่นที่ใช้วิธีนี้ เช่น RethinkDB และ MongoDB

ข้อดีคือจำนวน partition ปรับตามปริมาณข้อมูลเอง ข้อเสียคือฐานข้อมูลใหม่ที่ยังว่างอยู่จะเริ่มต้นด้วย partition เดียว งานทั้งหมดจึงไปตกที่เครื่องเดียวในช่วงแรก บางระบบจึงให้แบ่ง partition ไว้ล่วงหน้า เรียกว่า pre-splitting

วิธีที่สาม จำนวน partition ตามจำนวนเครื่อง

Cassandra ใช้วิธีให้แต่ละเครื่องมีจำนวน partition คงที่ ค่าเริ่มต้นคือเครื่องละ 256 partition เมื่อเพิ่มเครื่องใหม่ เครื่องใหม่ก็สุ่มเลือก partition เดิมบางส่วนมาแบ่งครึ่ง แล้วรับครึ่งหนึ่งไปดูแล ขนาดของ partition จึงค่อนข้างคงที่ แม้ระบบจะโตขึ้น

ย้ายอัตโนมัติ หรือย้ายด้วยมือ

การย้ายข้อมูลแบบอัตโนมัติทั้งหมดฟังดูสะดวก แต่ผู้เขียนเตือนว่าอาจอันตราย

ลองนึกภาพเครื่องหนึ่งทำงานหนักจนตอบช้า เครื่องอื่นเข้าใจผิดว่ามันเสีย จึงเริ่มย้ายข้อมูลออกจากเครื่องนั้น การย้ายข้อมูลก็เพิ่มภาระให้เครือข่ายและเครื่องที่หนักอยู่แล้วเข้าไปอีก ปัญหาจึงลุกลามจนระบบล่มต่อกันเป็นลูกโซ่

ผู้เขียนจึงเห็นว่าการให้คนมีส่วนตัดสินใจเป็นเรื่องดี เช่น ให้ระบบเสนอแผนการย้ายมาก่อน แล้วให้ผู้ดูแลกดยืนยัน แม้จะช้ากว่า แต่ช่วยป้องกันเหตุไม่คาดฝันได้

ตอนที่ 8: ส่งคำขอไปให้ถูกเครื่อง

เมื่อข้อมูลกระจายอยู่หลายเครื่อง และย้ายไปมาได้ คำถามคือ ผู้ใช้จะรู้ได้อย่างไรว่าต้องส่งคำขอไปเครื่องไหน ปัญหานี้เรียกว่า service discovery

มีสามแนวทาง

แนวทางแรก ส่งคำขอไปเครื่องไหนก็ได้ ถ้าเครื่องนั้นมีข้อมูล ก็ตอบเลย ถ้าไม่มี ก็ส่งต่อไปเครื่องที่มี

แนวทางที่สอง มีชั้นกลางทำหน้าที่จัดเส้นทาง คำขอทั้งหมดผ่านชั้นนี้ก่อน ชั้นนี้รู้ว่าข้อมูลอยู่ที่ไหน แล้วส่งต่อให้ถูกเครื่อง

แนวทางที่สาม ให้ฝั่งผู้ใช้รู้เองว่าข้อมูลอยู่เครื่องไหน แล้วติดต่อเครื่องนั้นตรง ๆ

ทุกแนวทางเจอปัญหาเดียวกัน คือ เมื่อข้อมูลย้าย ทุกฝ่ายต้องรู้ข่าวพร้อมกันอย่างถูกต้อง

หลายระบบแก้ด้วยบริการกลางชื่อ ZooKeeper ซึ่งคอยจดว่า partition ไหนอยู่เครื่องไหน เมื่อมีการเปลี่ยนแปลง ZooKeeper ก็แจ้งชั้นจัดเส้นทางให้อัปเดตตาม ระบบที่ใช้วิธีนี้ เช่น HBase, SolrCloud และ Kafka ส่วน MongoDB มีระบบคล้ายกันเป็นของตัวเอง

Cassandra และ Riak ใช้วิธีต่างออกไป คือให้เครื่องต่าง ๆ บอกต่อข่าวกันเอง คล้ายการซุบซิบปากต่อปาก วิธีนี้เรียกว่า gossip protocol ทำให้ไม่ต้องพึ่งบริการกลาง แต่ตัวฐานข้อมูลเองก็ซับซ้อนขึ้น

การประมวลผลคำสั่งซับซ้อน

ที่ผ่านมาเป็นคำของ่าย ๆ ที่อ่านหรือเขียนคีย์เดียว แต่ฐานข้อมูลสำหรับงานวิเคราะห์ ที่เรียกว่า MPP ย่อมาจาก Massively Parallel Processing จะแตกคำสั่งที่ซับซ้อน เช่นการเชื่อมตารางหรือการคำนวณผลรวม ออกเป็นงานย่อย แล้วกระจายให้หลายเครื่องช่วยกันทำพร้อมกัน เรื่องนี้จะพูดถึงอีกครั้งในบทที่สิบ

ตอนที่ 9: สรุปท้ายบท

มาทบทวนกันอีกครั้งครับ

หนึ่ง เราแบ่งข้อมูลเมื่อข้อมูลหรือภาระงานใหญ่เกินกว่าเครื่องเดียวจะรับไหว เป้าหมายคือกระจายให้เท่ากัน และหลีกเลี่ยงจุดร้อน

สอง การแบ่งตามช่วงของคีย์ ค้นหาเป็นช่วงได้ดี แต่เสี่ยงต่อจุดร้อน ส่วนการแบ่งตามค่าแฮช กระจายได้เท่ากันกว่า แต่เสียความสามารถในการค้นหาเป็นช่วง

สาม ดัชนีรองแบ่งได้สองแบบ แบบประจำแต่ละ partition เขียนง่าย แต่การอ่านต้องถามทุก partition ส่วนแบบดัชนีรวม อ่านได้เร็ว แต่การเขียนต้องแก้หลายที่

สี่ การย้ายข้อมูลเมื่อเพิ่มหรือลดเครื่อง ไม่ควรใช้การหารด้วยจำนวนเครื่อง แต่ควรใช้ partition จำนวนคงที่ แบ่งแบบยืดหยุ่น หรือแบ่งตามจำนวนเครื่อง และควรให้คนมีส่วนตัดสินใจด้วย

ห้า การส่งคำขอไปให้ถูกเครื่อง ทำได้หลายวิธี และหลายระบบใช้บริการกลางอย่าง ZooKeeper คอยติดตามว่าข้อมูลอยู่ที่ไหน

ในบทนี้ เราดูแค่การเขียนที่แตะ partition เดียว แต่ถ้าการเขียนครั้งหนึ่งต้องแก้หลาย partition พร้อมกัน แล้วบาง partition สำเร็จ บาง partition ล้มเหลว จะเกิดอะไรขึ้น นั่นคือเรื่องของบทถัดไป

บทต่อไปคือบทที่เจ็ด ว่าด้วยธุรกรรม หรือ Transactions ขอบคุณที่ฟังครับ

ทิปสำหรับ Edge: ในตอนที่ 7 ผมพิมพ์ขนาดของ HBase พลาดเป็น "กิโลไบต์" แล้วแก้กลางประโยค ถ้าจะใช้ฟังจริง แนะนำให้ลบคำว่า "10 กิโลไบต์ ขอแก้นะครับ" ออก ให้เหลือแค่ "10 กิกะไบต์" ครับ ส่วนตัวอย่างรถมือสองในตอนที่ 6 เป็นตอนที่เข้าใจยากที่สุดของบทนี้ ถ้าฟังรอบแรกแล้วยังไม่เข้าใจ ลองฟังเฉพาะตอนนั้นซ้ำอีกรอบได้ครับ

จะให้ทำบทที่ 7 เรื่อง Transactions ต่อเลยไหมครับ หรือจะรวมบทที่ 1–6 เป็นไฟล์ PDF ไว้เปิดใน Edge ก็ได้ 😊